In [ ]:
```
import marimo as mo
import numpy as np
import matplotlib.pyplot as plt
```

Модель Ходжкина — Хаксли

Интерактивная дискретная модель с ползунками.

$$
\begin{aligned}
C_m \dot{V} &= I_{\text{ext}} - \bar{g}_{\text{Na}}\, m^3 h\, (V - E_{\text{Na}}) - \bar{g}_{\text{K}}\, n^4\, (V - E_{\text{K}}) - \bar{g}_{\text{L}}\, (V - E_{\text{L}}), \\
\dot{m} &= \alpha_m(V)\,(1 - m) - \beta_m(V)\, m, \\
\dot{h} &= \alpha_h(V)\,(1 - h) - \beta_h(V)\, h, \\
\dot{n} &= \alpha_n(V)\,(1 - n) - \beta_n(V)\, n.
\end{aligned}
$$

где:

- V — мембранный потенциал;
- m, h, n — гейтовые переменные;
- C_m — ёмкость мембраны;
- $E_{\text{Na}}, E_{\text{K}}, E_{\text{L}}$ — реверсивные потенциалы;
- $\bar{g}_{\text{Na}}, \bar{g}_{\text{K}}, \bar{g}_{\text{L}}$ — максимальные проводимости.

In [ ]:
```
I_ext = mo.ui.slider(0.0, 30.0, 0.1, value=10.0, label="I_ext (мкА/см²)")
g_Na  = mo.ui.slider(0.0, 200.0, 1.0, value=120.0, label="ḡ_Na (мСм/см²)")
g_K   = mo.ui.slider(0.0, 100.0, 1.0, value=36.0,  label="ḡ_K (мСм/см²)")
g_L   = mo.ui.slider(0.0, 5.0,  0.05, value=0.3,  label="ḡ_L (мСм/см²)")
```

In [ ]:
```
controls = mo.vstack([
    mo.md("### Параметры модели"),
    I_ext, g_Na, g_K, g_L,
])
controls
```

Параметры модели

I_ext (мкА/см²)

ḡ_Na (мСм/см²)

ḡ_K (мСм/см²)

ḡ_L (мСм/см²)

In [ ]:
```
# --- фиксированные параметры мембраны ---
C_m   = 1.0          # мкФ/см²
E_Na  = 50.0         # мВ
E_K   = -77.0        # мВ
E_L   = -54.387      # мВ

# --- шаг интегрирования (модель жёсткая!) ---
dt = 0.01            # мс
```

```
N  = 50000           # 500 мс модельного времени
n  = np.arange(N)
t  = n * dt

# --- массивы решения ---
V = np.zeros(N)
m = np.zeros(N)
h = np.zeros(N)
nn = np.zeros(N)     # имя n занято под массив времени

# --- начальные условия (покой HH) ---
V[0]  = -65.0
m[0]  = 0.0529
h[0]  = 0.5961
nn[0] = 0.3177

# --- читаем значения ползунков ---
I_val   = I_ext.value
gNa_val = g_Na.value
gK_val  = g_K.value
gL_val  = g_L.value

# --- функции скоростей (с защитой от деления на 0) ---
def alpha_m(V):
    x = V + 40.0
    return np.where(np.abs(x) < 1e-7, 1.0, 0.1 * x / (1.0 - np.exp(-x /
10.0)))

def beta_m(V):
    return 4.0 * np.exp(-(V + 65.0) / 18.0)

def alpha_h(V):
    return 0.07 * np.exp(-(V + 65.0) / 20.0)

def beta_h(V):
    return 1.0 / (1.0 + np.exp(-(V + 35.0) / 10.0))

def alpha_n(V):
    x = V + 55.0
    return np.where(np.abs(x) < 1e-7, 0.1, 0.01 * x / (1.0 - np.exp(-x /
10.0)))

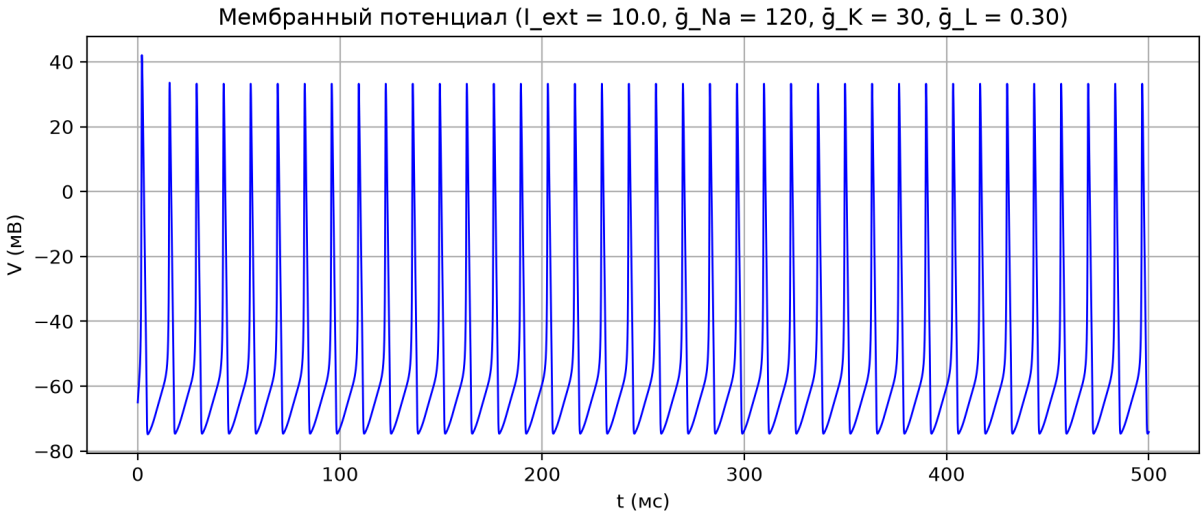
def beta_n(V):
    return 0.125 * np.exp(-(V + 65.0) / 80.0)

# --- итерации (метод Эйлера) ---
for k in range(N - 1):
    I_Na = gNa_val * m[k]**3 * h[k] * (V[k] - E_Na)
    I_K  = gK_val  * nn[k]**4       * (V[k] - E_K)
    I_L  = gL_val                   * (V[k] - E_L)

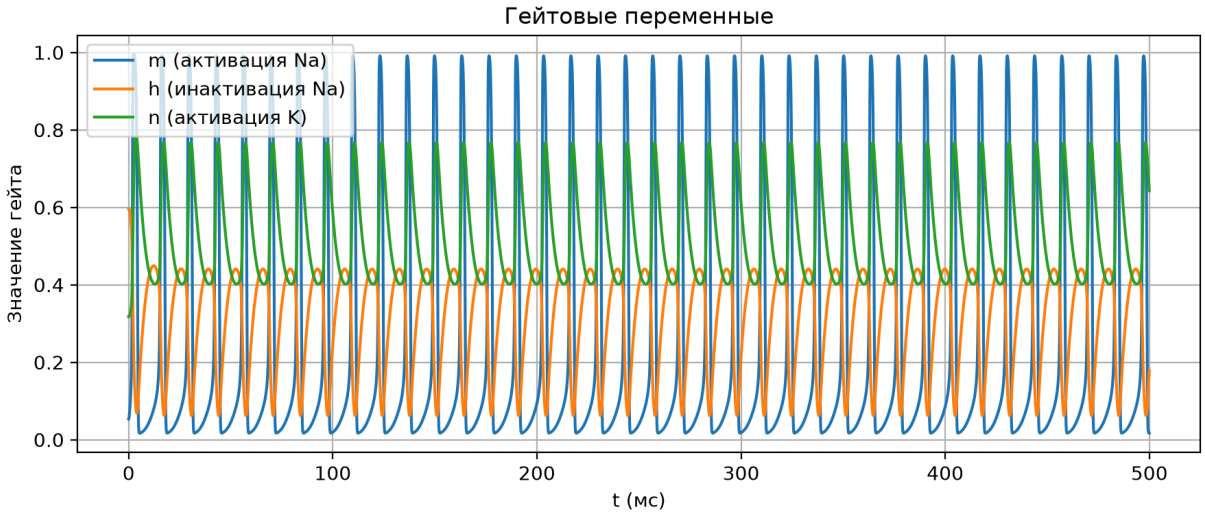
    dV = (I_val - I_Na - I_K - I_L) / C_m
    dm = alpha_m(V[k]) * (1.0 - m[k])  - beta_m(V[k]) * m[k]
    dh = alpha_h(V[k]) * (1.0 - h[k])  - beta_h(V[k]) * h[k]
    dn = alpha_n(V[k]) * (1.0 - nn[k]) - beta_n(V[k]) * nn[k]
```

```
    V[k + 1]  = V[k]  + dt * dV
    m[k + 1]  = m[k]  + dt * dm
    h[k + 1]  = h[k]  + dt * dh
    nn[k + 1] = nn[k] + dt * dn
```

In [ ]:
```
fig1, ax1 = plt.subplots(figsize=(9, 4))
ax1.plot(t, V, 'b', lw=1.0)
ax1.set_xlabel('t (мс)')
ax1.set_ylabel('V (мВ)')
ax1.set_title(f'Мембранный потенциал '
              f'(I_ext = {I_val:.1f}, ḡ_Na = {gNa_val:.0f}, '
              f'ḡ_K = {gK_val:.0f}, ḡ_L = {gL_val:.2f})')
ax1.grid(True)
fig1.tight_layout()
fig1
```



In [ ]:
```
fig2, ax2 = plt.subplots(figsize=(9, 4))
ax2.plot(t, m,  label='m (активация Na)')
ax2.plot(t, h,  label='h (инактивация Na)')
ax2.plot(t, nn, label='n (активация K)')
ax2.set_xlabel('t (мс)')
ax2.set_ylabel('Значение гейта')
ax2.set_title('Гейтовые переменные')
ax2.legend()
ax2.grid(True)
fig2.tight_layout()
fig2
```



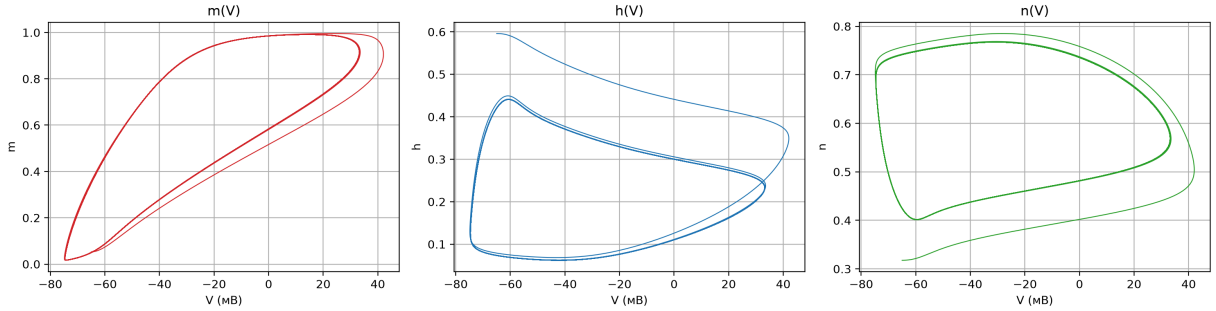
In [ ]:
```
fig3, axes3 = plt.subplots(1, 3, figsize=(15, 4))

axes3[0].plot(V, m, 'tab:red', lw=0.8)
axes3[0].set_xlabel('V (мВ)'); axes3[0].set_ylabel('m')
axes3[0].set_title('m(V)'); axes3[0].grid(True)

axes3[1].plot(V, h, 'tab:blue', lw=0.8)
axes3[1].set_xlabel('V (мВ)'); axes3[1].set_ylabel('h')
axes3[1].set_title('h(V)'); axes3[1].grid(True)

axes3[2].plot(V, nn, 'tab:green', lw=0.8)
axes3[2].set_xlabel('V (мВ)'); axes3[2].set_ylabel('n')
axes3[2].set_title('n(V)'); axes3[2].grid(True)

fig3.tight_layout()
fig3
```



In [ ]:
```
fig, axes = plt.subplots(2, 1, figsize=(8, 8), sharex=True)

axes[0].plot(t, V, 'b', lw=1.0)
axes[0].set_ylabel('V (мВ)')
axes[0].set_title('Мембранный потенциал')
axes[0].grid(True)

axes[1].plot(t, m,  label='m')
axes[1].plot(t, h,  label='h')
axes[1].plot(t, nn, label='n')
axes[1].set_xlabel('t (мс)')
axes[1].set_ylabel('Гейты')
```

```
axes[1].set_title('Гейтовые переменные')
axes[1].legend()
axes[1].grid(True)

fig.tight_layout()

mo.hstack([controls, fig], justify="start", gap=2)
```

Параметры модели

I_ext (мкА/см²)

ḡ_Na (мСм/см²)

ḡ_K (мСм/см²)

ḡ_L (мСм/см²)

